

Assignment 3 Milestone 3

Group members: Kevin Jagadeesh (*kavinj*), Jessica Chen (*haowenc9*)

Our application utilizes an inverted index, a bm25 and tf idf scoring system, can bring the top results from top 1 to top 50 if need be, PR weight can be changed from 0 to 1, there are hybrid and strict modes, a page ranking system that can be check marked, and a search bar with a search button.

24 Test Queries

Query Criteria

- entity/name search
 - People, programs, organizations
- Short keyword search
 - 1-2 terms
- Longer topical search
 - 3-4 terms
- Academic intent search
 - Admissions, labs, courses, research areas
- Queries likely to fail w/ strict AND
 - missing/rare term risk

Query Evaluation Criteria

- **Response Time:** *how fast the queries are processed and load the results*
- **Ranking Quality:** *how relevant were the top k results of the query*
 - *Good (top result is the most relevant page)*
 - *Acceptable (top results are related but not perfectly ordered)*
 - *Poor (results are off topic or too few results appear)*

Testing Unsuitable Queries

These are 10 poor queries to show any limitations the program may have (long/natural language, ambiguous, tail, and/or phrase-like queries which may or may not have rare terms).

1. Information retrieval project
 - ◆ Multi-term topical query; easy to miss one term in strict AND
2. Graduate student forms
 - ◆ Administrative intent w/ lexical variation; forms/documents

3. Machine learning systems
 - ◆ Topic query where relevant docs may contain only partial term overlap
4. Data science seminar
 - ◆ Event-style query; sparse co-occurrences of all terms
5. Software engineering requirements
 - ◆ Program policy query w/ long-tail wording
6. Database systems lab
 - ◆ Multi-term research intent, partial match sensitivity
7. Cybersecurity club meetings
 - ◆ Activity query; some pages only mention a subset of terms
8. Computer vision deep learning
 - ◆ Long topical query likely to fragment across pages
9. Natural language processing course
 - ◆ Course-intent query w/ wording variation
10. Distributed systems research
 - ◆ Broad technical query w/ many near matches
11. Human computer interaction
 - ◆ Phrase query that may split across tokenization variants
12. Bioinformatics phd admissions
 - ◆ Specific academic intent query w/ sparse text overlap

Response Time: Response time ranged from 1ms to 103ms across all 24 queries (average: 28ms), all well under the 300ms limit.

Ranking Quality: acceptable overall; weaker on sparse long-tail queries (e.g., “bioinformatics phd admissions”).

Acceptable Test Queries

These are 10 short keywords and frequent topic keywords to show the efficiency and responsiveness of the program.

1. Cristina lopes
 - ◆ Strong name/entity query
2. Machine learning
 - ◆ Common 2 term intent w/ high corpus support
3. ACM
 - ◆ Specific acronym/entity query
4. Master of software engineering
 - ◆ Program name w/ strong exact matches
5. lcs helpdesk
 - ◆ Specific service/entity query

6. Uci informatics
 - ◆ High freq. Institutional entity query
7. Graduate admissions
 - ◆ Common administrative intent
8. Faculty directory
 - ◆ Stable navigational query
9. Undergrad forms
 - ◆ Stable administrative query
10. Tutoring center
 - ◆ Specific service query
11. Honors program
 - ◆ Program name style query
12. Research labs
 - ◆ Broad but common institutional query

Response Time: Response time ranged from 1ms to 103ms across all 24 queries (average: 28ms), all well under the 300ms limit.

Ranking Quality: acceptable

Test Cases that are Notable

“Bioinformatics phd admissions”



Information Retrieval Search

Web interface powered by your local index.

bioinformatics phd admissions

Mode Ranking Top K PR Weight

☒ use PageRank

Search

1 result(s)

#1 score=0.800068 matched=3 time_ms=16.616
<https://www.cs.uci.edu/news/page/3/>

Corpus sparsity means that in a large collection of documents, most words show up in only a handful of places, and each document uses just a small slice of the overall vocabulary. If you imagine a giant table showing which words appear in which documents, almost every cell in that table would be empty. This happens because language naturally has a few very common words and an enormous number of rare, specific ones.

For search engines, this sparsity is both helpful and tricky. It makes storage efficient because you only need to keep track of the few places where each word actually appears. But it also makes ranking harder for queries that use rare terms, reduces the chance that related documents share the same vocabulary, and makes it tougher to match meaning rather than just exact words. Techniques like stemming, BM25, and query expansion are designed to soften these problems and make retrieval more reliable despite the sparsity.

What was changed in the search engine

1. **Hybrid retrieval fallback (search.py)**
 - Strict AND is attempted first.
 - If strict AND returns no candidates, fallback requires high term overlap first, then relaxes gradually.
2. **BM25 ranking + coordination (search.py)**
 - Replaced the prior log-TF*IDF ranking.
 - Added a coordination factor favoring documents matching more query terms.
3. **Lexicon-seek postings lookup (indexer.py and search.py)**
 - Built `final\_index/lexicon.tsv` with term byte offsets and document frequencies.
 - Query execution now seeks directly to postings instead of scanning the entire index file.
4. **Small-memory indexing path (indexer.py)**
 - Partial indexes are merged with streaming k-way merge.
 - Avoids loading all partial postings into memory at once.

Why these changes address the poor queries

- **Effectiveness (recall):** hybrid fallback recovers relevant docs when strict AND is too brittle.
- **Effectiveness (ranking):** BM25 with coordination improves ordering for longer multi-term intents.
- **Efficiency (runtime):** lexicon-based seeking avoids full-index scans per query.

- **Memory footprint:** streaming merge keeps indexing memory usage bounded relative to index size.

Reproducibility / Demo Commands

1. Rebuild index:

- a. ``python indexer.py``
- b. *Start off by running the indexer.py this can take possibly 20 minutes or longer*

2. Compare strict vs hybrid for one query:

- a. ``python search.py --query "distributed systems research" --mode strict --topk 5``
- b. ``python search.py --query "distributed systems research" --mode hybrid --topk 5``
- c. *A search bar and familiar form type filters (mode, ranking, top k, PR weight, use PageRank) will be available to select on the webpage once the application is launched in a port*

3. Run with web interface (optional, recommended)

- a. *Start server: `python web_app.py`*
- b. *Open: <http://127.0.0.1:5000>*
- c. *Available controls: mode, ranking (TF-IDF/BM25), top-k, PageRank weight, and PageRank on/off*

4. Batch evaluation output (24 queries):

- a. ``milestone3_results.json``
- b. *The results of the tested queries are available in the json file mentioned above*

***Note: we recommend testing for the top 5 results unless you wish to explicitly evaluate recall at large cutoffs